

Universidade Federal de Goiás
Instituto de Informática
Compiladores e Compiladores 1
2026-1

Trabalho 2 - Implementação Compilador para a Linguagem
G-V2

Prof. Thierson Couto Rosa

1 Objetivo

Este trabalho tem como objetivo a implementação de um compilador didático para a linguagem G-V2. A linguagem G-V2 é uma extensão da linguagem G-V1 do trabalho T1 que inclui funções e o tipo vetor. A gramática para G-V2 mantém uma estrutura semelhante a G-V1. Basicamente as mudanças estão nas produções iniciais, relacionadas a declarações que agora incluem declarações de variáveis globais, declarações de funções e o tipo vetor. Algumas mudanças pontuais também são necessárias na produção relacionada a expressões, pois variáveis do tipo vetor necessitam usar um índice. Os tipos básicos da linguagem continuam sendo *car* e *int*. Os vetores podem ser apenas vetores desses tipos básicos.

2 Gramática para a linguagem G-V2

A seguir, é apresentada a gramática para a linguagem G-V2. Não será apresentada a quádrupla completa correspondente à gramática que descreve a linguagem. O conjunto V_N de símbolos não-terminais da gramática é formado pelo conjunto de símbolos nas regras de derivação que iniciam com uma letra maiúscula e que aparecem do lado esquerdo da seta em alguma produção.

O conjunto V_T de terminais da gramática é formado por todos os símbolos escritos completamente em letras maiúscula e por símbolos que aparecem entre apóstrofes.

<i>Programa</i>	→ <i>DeclVarGlobais DeclFunc DeclPrograma</i>
<i>DeclVarGlobais</i>	→ GLOBAL <i>VarSection</i> ε
<i>VarSection</i>	→ ' [' <i>ListaDeclVar</i> '] '
<i>ListaDeclVar</i>	→ <i>ListVar</i> ':' ' <i>Tipo</i> ';' <i>ListaDeclVar</i> <i>ListVar</i> ':' ' <i>Tipo</i> ';' '
<i>ListVar</i>	→ IDENTIFICADOR ' , ' <i>ListVar</i> IDENTIFICADOR ' [' INTCONST '] ' ' , ' <i>ListVar</i> IDENTIFICADOR IDENTIFICADOR ' [' INTCONST '] '
<i>DeclFunc</i>	→ FUNCAO ' [' IDENTIFICADOR ' (' <i>ListaParametros</i> ') ' ' : ' <i>Tipo</i> <i>Bloco</i> <i>ListaFuncoes</i> '] ' ' ε
<i>ListaFuncoes</i>	→ IDENTIFICADOR ' (' <i>ListaParametros</i> ') ' ' : ' <i>Tipo</i> <i>Bloco</i> <i>ListaFuncoes</i> ε
<i>ListaParametros</i>	→ <i>ListaParametrosTail</i> ε
<i>ListaParametrosTail</i>	→ IDENTIFICADOR ':' ' <i>Tipo</i> IDENTIFICADOR ' [' '] ' ' : ' <i>Tipo</i> IDENTIFICADOR ':' ' <i>Tipo</i> ' , ' <i>ListaParametrosTail</i> IDENTIFICADOR ' [' '] ' ' : ' <i>Tipo</i> ' , ' <i>ListaParametrosTail</i>
<i>DeclPrograma</i>	→ PRINCIPAL <i>Bloco</i>
<i>Bloco</i>	→ ' { ' <i>ListaComando</i> ' } ' <i>VarSection</i> ' { ' <i>ListaComando</i> ' } '
<i>Tipo</i>	→ INT CAR
<i>ListaComando</i>	→ <i>Comando</i> <i>Comando</i> <i>ListaComando</i>

<i>Comando</i>	→ ' ; '
	<i>Expr</i> ' ; '
	RETORNE <i>Expr</i> ' ; '
	LEIA <i>LValueExpr</i> ' ; '
	ESCREVA <i>Expr</i> ' ; '
	ESCREVA CADEIACARACTERES ' ; '
	NOVALINHA ' ; '
	SE ' (' <i>Expr</i> ') ' ENTÃO <i>Comando</i> FIMSE
	SE ' (' <i>Expr</i> ') ' ENTÃO <i>Comando</i> SENÃO <i>Comando</i> FIMSE
	ENQUANTO ' (' <i>Expr</i> ') ' <i>Comando</i>
	<i>Bloco</i>
<i>Expr</i>	→ <i>LValueExpr</i> ATRIBUICAO <i>Expr</i>
	<i>OrExpr</i>
<i>LValueExp</i>	→ IDENTIFICADOR ' [' <i>Expr</i> '] '
	IDENTIFICADOR
<i>OrExpr</i>	→ <i>OrExpr</i> OU <i>AndExpr</i>
	<i>AndExpr</i>
<i>AndExpr</i>	→ <i>AndExpr</i> E <i>EqExpr</i>
	<i>EqExpr</i>
<i>EqExpr</i>	→ <i>EqExpr</i> IGUAL <i>DesigExpr</i>
	<i>EqExpr</i> DIFERENTE <i>DesigExpr</i>
	<i>DesigExpr</i>
<i>DesigExpr</i>	→ <i>DesigExpr</i> '<' <i>AddExpr</i>
	<i>DesigExpr</i> '>' <i>AddExpr</i>
	<i>DesigExpr</i> MAIORIGUAL <i>AddExpr</i>
	<i>DesigExpr</i> MENORIGUAL <i>AddExpr</i>
	<i>AddExpr</i>
<i>AddExpr</i>	→ <i>AddExpr</i> '+' <i>MulExpr</i>
	<i>AddExpr</i> '-' <i>MulExpr</i>
	<i>MulExpr</i>
<i>MulExpr</i>	→ <i>MulExpr</i> '*' <i>UnExpr</i>
	<i>MulExpr</i> '/' <i>UnExpr</i>
	<i>UnExpr</i>
<i>UnExpr</i>	→ '-' <i>PrimExpr</i>
	'!' <i>PrimExpr</i>
	<i>PrimExpr</i>

$$\begin{aligned}
PrimExpr \rightarrow & \text{IDENTIFICADOR } '(' ListExpr ')', \\
& | \text{IDENTIFICADOR } '(' ' ' ')', \\
& | \text{IDENTIFICADOR } '[' Expr ']', \\
& | \text{IDENTIFICADOR} \\
& | \text{CARCONST} \\
& | \text{INTCONST} \\
& | '(' Expr ')',
\end{aligned}$$

$$\begin{aligned}
ListExpr \rightarrow & Expr \\
& | ListExpr ', ' Expr
\end{aligned}$$

A seguir são apresentadas as definições de tokens que não correspondem a um único caracter. A primeira coluna do tabela 1 corresponde ao *token* e a segunda coluna corresponde à definição dos lexemas que podem aparecer no *token* correspondente à primeira coluna do quadro. Os alunos devem criar expressões regulares no arquivo de entrada para o Flex de modo que o analisador léxico retorne o token e o lexema correspondente encontrado na entrada.

Token	Definição
GLOBAL	global
FUNCAO	funcao
PRINCIPAL	principal
IDENTIFICADOR	Inicia com letra ou '_' seguido por zero ou mais: letras, dígitos ou '_'.
INT	int
CAR	car
RETORNE	retorne
LEIA	leia
ESCREVA	escreva
NOVALINHA	novalinha
SE	se
ENTAO	entao
SENAO	senao
FIMSE	fimse
ENQUANTO	enquanto
CADEIACARACTERES	Inicia e termina com " e pode ter qualquer caractere exceto \n.
OU	
E	&
IGUAL	==
DIFERENTE	!=
MAIORIGUAL	>=
MENORIGUAL	<=
CARCONST	Caractere delimitado por '. Inclui caracteres com escape: '\n', '\t', etc.
INTCONST	Constante dos números inteiros, podendo ou ter sinal (+ ou -) ou não.

Tabela 1: Os tokens que não correspondem a apenas um caractere na linguagem G-V2. A tabela mostra o nome do token na primeira coluna e o padrão de formação dos lexemas na segunda coluna.

3 Geração dos analisadores sintático e léxico para a linguagem G-V2

Devem ser criados os arquivos de entrada para o Bison e o Flex. Os arquivos devem ser compilados e ligados em um único executável denominado `g-v2`.

3.1 Analisador Léxico

Poucas mudanças precisam ser feitas no analisador léxico para G-V2, em relação ao analisador léxico para GV1. Especificamente, o arquivo fonte para o Flex deve incluir os seguintes *tokens*: `'[', ']', funcao, global, retorne`, veja a tabela 1.

Recomenda-se que os alunos façam uma cópia do arquivo `g-v1.l` utilizado como entrada para o Flex no trabalho T1. Essa cópia pode ser transformada no arquivo `g-v2.l` que dará origem ao analisador léxico da linguagem G-V2. As mesmas questões relacionadas a tratamento de comentários e de cadeias de caracteres especificadas em G-V1 continuam valendo em G-V2, portanto o código feito para esse tratamento pode ser completamente reutilizado no arquivo de entrada do Bison para a linguagem G-V2.

Material de apoio

- Slides da aula sobre o Flex disponíveis no Bloco 1 da Plataforma Turing.
- Gnu Flex manual
- Pasta compactada com o exemplo de entradas para o Bison e Flex disponível no Bloco 2 da Plataforma Turing.

3.2 Analisador Sintático

A linguagem G-V2 se assemelha à linguagem G-V1, mas se difere da última nos seguintes aspectos:

- Permite a definição de variáveis globais a todas as funções e à função **principal**. As variáveis globais devem ser declaradas antes de todas as funções, inclusive antes da função **principal**. A seção de declaração de variáveis globais inicia-se com a palavra reservada **global**, seguida por `'['` seguida por uma ou mais lista de declarações de variáveis. A seção de declaração de variáveis globais termina com o delimitador `']'`. A seção é omitida caso não haja variáveis globais no programa.
- Permite a declaração de funções. Uma seção de declaração de funções deve ser criada no início do arquivo ou logo após a seção de declaração de variáveis globais, caso essas existam. A seção inicia com a palavra-chave **funcao** seguida pelo delimitador `'['` e por uma ou mais declarações de funções. A seção termina com o delimitador `']'`. A seção é omitida caso não haja declarações de funções no programa.
- A seção de declaração de variáveis em um bloco na G-V1 iniciava e terminava com os delimitadores `'{'` e `'}'`. Na linguagem G-V2 esses delimitadores foram substituídos, respectivamente por `'['` e `']'`.
- Os tipos vetor de caracteres e vetor de inteiros são permitidos em G-V2.

A gramática da linguagem G-V2 adapta a gramática da linguagem G-V1 para acomodar as alterações apresentadas acima.

Recomenda-se que os alunos aproveitem o arquivo de entrada para o Bison (g-v1.y) da linguagem G-V1 como base para gerar o arquivo entrada correspondente à linguagem G-V2. A maior parte das ações que envolvem a pilha semântica do Bison, utilizadas para a criação da AST também poderão ser aproveitadas. Contudo, os tipos de dados dos nós da AST devem ser adaptados para conter funções, com suas listas de parâmetro, seus tipos e blocos.

Material de apoio

- Slides da aula sobre o Bison disponíveis no Bloco 1 da Plataforma Turing.
- Gnu Bison Manual
- Pasta compactada com o exemplo de entradas para o Bison e Flex disponível no Bloco 2 da Plataforma Turing.

4 Geração da representação intermediária do programa de entrada

A representação intermediária de um programa fonte em G-V2 corresponde a uma árvore sintática abstrata. As estruturas de dados utilizadas para armazenar a AST em memória, criadas para a linguagem G-V1 devem ser adaptadas para a AST da linguagem G-V2. Poucas alterações serão necessárias. Especificamente, os nós da AST devem ser alterados de modo que sejam capazes de representar funções com seus nomes, listas de parâmetros, tipo e bloco. Além disso, um nó da AST deve ser capaz de representar um identificador de vetor e a expressão que representa seu índice.

Material de apoio

- Slides da aula sobre esquema de tradução disponíveis no Bloco 2 da Plataforma Turing.
- O exemplo de construção de árvore sintática abstrata apresentado na Seção 2.8 do livro-texto para a linguagem SimpleLang.
- Slides e pelos códigos fornecidos pelo professor (disponíveis no Bloco 2 na Turing) para o referido exemplo.
- Capítulo 6 do livro *Introduction to Compilers and Language Design*¹

5 Tabela de Símbolos para a linguagem G-V2

Conforme mostra a descrição da gramática, em G-V2 pode haver variáveis que são globais a todo o programa. Um bloco pode conter uma lista de declaração de variáveis locais (que pode ser vazia) e pode ou não ter uma lista de comandos. Porém, um comando da lista de comandos pode ser um outro bloco. Ou seja, os blocos aninham-se uns dentro dos outros. Além disso, cada bloco que possui uma declaração de variáveis locais inicia um escopo novo.

¹<https://dthain.github.io/books/compiler/>

No caso de funções, os identificadores que formam os seus parâmetros estão no mesmo escopo do bloco da função. Essa situação permite a coexistência de variáveis distintas que possuem o mesmo lexema de identificador (mesmo nome), por exemplo um mesmo nome x declarado em um escopo mais externo e outro objeto de nome x declarado em um escopo mais interno. Veja explicações mais detalhadas no slide sobre tabelas de símbolos na Plataforma Turing.

Nesta parte do trabalho deve ser implementada uma pilha de tabela de símbolos (ou tabela de lexemas de identificadores) capaz de armazenar lexemas de identificador. Essa estrutura deve ter associada a ela o seguinte conjunto de operações:

- a) iniciar a pilha de tabela de símbolos - operação que cria uma estrutura de dados inicialmente vazia para representar a pilha de tabelas de símbolos (pilha de escopos);
- b) criar uma nova tabela de símbolos (novo escopo) e empilhá-la na pilha de tabela de símbolos.
- c) pesquisar por um nome (lexema de identificador) na pilha de tabelas de símbolos. Essa operação deve iniciar a pesquisa na tabela que está no topo da pilha (tabela de símbolos correspondente ao último escopo criado) e enquanto não encontrar, procurar nas tabelas seguintes, do topo em direção à base da pilha de tabelas de símbolos. Deve retornar um ponteiro para a entrada da tabela de símbolos onde o nome foi encontrado ou um ponteiro vazio, caso o nome procurado não se encontre na pilha de tabelas.
- d) remover escopo atual - essa função elimina a tabela de símbolos que está no topo da pilha de tabelas.
- e) inserir um nome de função na tabela de símbolos atual (que está no topo da pilha de tabelas). Essa operação deve conter como parâmetros: a string com o nome da função, um número inteiro correspondente ao número de argumentos da função, o tipo retornado pela função (os dois últimos parâmetros serão úteis para as fases de análise semântica e/ou geração de código do compilador). Se a função tiver argumentos, será necessário criar um esquema especial para armazenar informações sobre o lexema correspondente ao identificador de cada argumento. Nesse caso, a entrada da tabela de símbolos que contém o nome de uma função deve conter essa estrutura especial, ou ter um ponteiro que contenha o endereço dessa estrutura.

observação: *Os lexemas de argumentos de uma função têm uma característica distinta dos demais lexemas nas tabelas de símbolos. Eles não podem ser acessíveis em escopos externos à função. Por outro lado, em uma chamada à função que pode ser externa à função (chamada não recursiva), o compilador precisa ter acesso aos argumentos para checar se os tipos dos argumentos reais que aparecem na chamada correspondem aos tipos dos argumentos formais da função. Portanto, há um quase-dilema nesse caso: os lexemas dos argumentos não podem ser vistos pelo código do usuário que está em um escopo externo à função. Por outro lado, o compilador precisa ter acesso aos lexemas dos argumentos e a seus tipos para fazer a checagem de tipos durante uma chamada. Essa é a razão pela qual os argumentos precisam ser tratados de forma distinta dos demais identificadores na pilha de tabelas.*

- f) inserir um nome de variável na tabela de símbolos atual. Essa operação deve ter como parâmetros: a string com o nome da variável, um inteiro indicando o tipo da variável (informação a ser utilizada futuramente, na análise semântica) e um inteiro positivo,

indicando em qual posição da lista de declaração de variáveis a variável ocorreu (essa informação será útil posteriormente, para a geração de código).

g) inserir o nome de um parâmetro na tabela de símbolos atual. Essa operação deve ter como parâmetros: a string com o nome do parâmetro, um inteiro indicando o tipo do parâmetro (informação a ser utilizada futuramente, na análise semântica), um inteiro positivo, indicando em qual posição da lista de declaração de parâmetro o parâmetro ocorreu (essa informação será útil posteriormente, para a geração de código).

h) eliminar a pilha de tabelas de símbolos.

Material de apoio

- Slides da aula sobre tabelas de símbolos disponíveis no Bloco 2 da Plataforma Turing.
- Capítulo 7 do livro *Introduction to Compilers and Language Design*²

6 Analisador Semântico

Aspectos Semânticos da Linguagem G-V2

Declaração de variáveis

As regras de escopo e de tempo de vida das variáveis, da linguagem G-V2 são semelhantes às regras da linguagem C, porém há as seguintes restrições:

- Todas as variáveis declaradas na seção **global** '[' são globais às funções declaradas após as declarações dessas variáveis e também são globais ao bloco principal. Essa seção, se existir, aparece no início do texto do programa fonte, antes da seção de declarações de funções ou da declaração da função principal. As variáveis globais só podem ser declaradas nessa seção que termina com o símbolo ']'.
- Não há o conceito de protótipo de função. Todas funções utilizadas no programa principal devem ser completamente declaradas (cabeçalho e corpo) na seção de declaração de funções que inicia com **fucao** '[' e termina com ']', antes da declaração **principal** que representa o bloco principal do programa.
- Não há o conceito de variáveis **static** como na linguagem C. O tempo de vida de uma variável local a um bloco corresponde à duração do bloco.
- Todo programa em G-V2 ocupa apenas um arquivo e não há compilação separada. Portanto, não há declarações do tipo **extern** como na linguagem C.
- Uma variável declarada em um bloco é local a esse bloco e global a todos os blocos internos a esse bloco, em qualquer profundidade de aninhamento de blocos. Entretanto, se uma variável local a um bloco tem o mesmo nome de uma variável global a esse bloco, a variável local se sobrepõe à variável global durante a duração do bloco.

²<https://dthain.github.io/books/compiler/>

- Os parâmetros formais têm escopo de variável local. Não é permitido uma variável local com mesmo nome de um parâmetro formal no bloco mais externo que forma a função. Entretanto, é permitido criar uma variável com mesmo nome de um parâmetro formal, em um bloco aninhado. Por exemplo:

```
int f(int a, int b){ int a, b; ....} -- não permitido.
int f(int a, int b) {....{int a,b; .....} .... } -- permitido
```

- A passagem de um tipo vetor como parâmetro é feita passando-se o endereço de início do vetor (passagem por referência) e qualquer alteração feita em algum elemento do vetor pela função é mantida no vetor passado pela chamada à função quando essa termina sua execução.

A declaração de qualquer nome deve preceder o seu uso em um escopo válido, isto é, um nome pode ser utilizado em um comando somente se ele tiver sido declarado previamente e se o comando estiver no escopo de sua declaração.

Tipos e Checagem de Tipos

A linguagem G-V2 possui apenas dois tipos básicos distintos: `int` e `car` e também os tipos vetor de `car` e vetor de `int`. A linguagem não permite operações entre objetos ou expressões de tipo básicos (`car` ou `int`) distintos ou entre objetos e expressões de tipo básicos e objetos do tipo vetor (`[]`). Contudo, são permitidas operações entre um elemento de um vetor e uma variável de tipo básico desde que sejam ambos do mesmo tipo básico (`int` ou `car`). O tipo vetor corresponde a um ponteiro para uma sequência de objetos de algum tipo básico. A atribuição de uma variável vetor `X` a outra variável vetor `Y` de mesmo tipo básico, corresponde à cópia do endereço da variável `X` para a variável `Y`. Essa atribuição é permitida somente se os vetores forem do mesmo tipo básico.

Ao contrário da linguagem C, na linguagem G-V2 o número de parâmetros formais de uma função e o número de parâmetros de uma chamada à função deve ser o mesmo. O tipo de cada parâmetro formal deve ser o mesmo de cada parâmetro de chamada correspondente. No caso em que o parâmetro de chamada corresponde a um elemento de um vetor, o parâmetro formal correspondente deve ser uma variável simples de mesmo tipo básico do vetor. Se a chamada passar um vetor como parâmetro, o parâmetro formal correspondente da função chamada também deve ser um vetor, sem o tamanho especificado (apenas o par de colchetes sem um valor entre eles). Além disso, o tipos básicos dos vetores do parâmetro formal e do parâmetro de chamada correspondente devem ser os mesmos.

Não existe o tipo lógico explícito na linguagem G-V2, entretanto nos comandos `se` e `enquanto` é necessário avaliar se uma expressão é verdadeira ou falsa para a tomada de uma decisão quanto ao fluxo de execução dos comandos internos a estes dois comandos. Assim como a linguagem C, a linguagem G-V1 trata como verdadeira uma expressão cujo valor é diferente de zero e como falsa, uma expressão cujo valor é zero. Obviamente, essa avaliação é possível somente em tempo de execução do programa fonte. Entretanto, a nível de análise semântica, uma expressão lógica (com operações OR, AND ou NEGAÇÃO) ou relacional (com operadores relacionais – `>`, `<`, `<=`, etc) devem receber tipo `int`.

Uma expressão retornada por uma função deve ter o mesmo tipo da função. A expressão do lado direito de uma atribuição deve ter o mesmo tipo da variável que recebe a expressão. Os operadores aritméticos devem ser aplicados em expressões do tipo `int`. Os operadores relacionais devem ser aplicados a operandos de mesmo tipo.

7 Geração de código

Se o programa de entrada estiver semanticamente correto, outro algoritmo de percurso deve ser iniciado na AST. Desta vez, o percurso será com o objetivo de gerar código. Nesse novo percurso, tabelas de símbolos devem ser criadas e empilhadas na pilha de tabelas à medida que o percurso encontra subárvores de declarações de variáveis. Também como ocorre durante o percurso de análise semântica, uma tabela é eliminada do topo da pilha ao final de uma subárvore correspondente a um bloco, se houve pelo menos uma declaração de variáveis nesse bloco. Como o programa está semanticamente correto, as informações na tabela não servem para checagem de escopo e tipo, mas sim, para identificar as posições de cada variável na memória e também o tamanho do espaço a ser alocado para cada variável (indicado pelo seu tipo).

Material de apoio

- Slides da aula sobre geração de código, disponíveis no Bloco 2 da Plataforma Turing.
- MIPS Assembly Language Programming (pdf disponível na Plataforma Turing).

8 Informações Sobre a Implementação e a Entrega

O trabalho pode ser desenvolvido em grupo de dois alunos e deve ser entregue até o dia 18/06/2026 via tarefa criada no site da disciplina na plataforma Turing. O trabalho deve ser apresentado durante as aulas dos dias 18/06/2026, 22/06/2026 e 25/06/2026 pela turma de Ciência da Computação e nos dias 18/06/2026, 23/06/2026 e 25/06/2026 pela turma de Engenharia de Computação. Os trabalhos são totalmente factíveis individualmente. Portanto, se um dos componentes abandonar o grupo, o outro deve continuar os trabalhos individualmente e apresentá-lo nas datas marcadas.

Deve ser entregue uma pasta compactada contendo:

- O arquivo de especificações para o gerador de analisador léxico utilizado (arquivo de entrada para o Flex/Lex - g-v2.l).
- O arquivo de especificações para o gerador de analisador sintático expandido com ações semânticas necessárias para a geração de árvore sintática abstrata (arquivo de entrada para o Yacc/Bison - g-v2.y).
- O código em C/C++ que implementa a tabela de símbolos e o arquivo headerfile correspondente.
- O código em C/C++ do analisador semântico que irá utilizar a árvore abstrata e a tabela de símbolos para detectar a ocorrência de erros semânticos em arquivos contendo programas escritos na linguagem Goianinha (e headerfile correspondente).
- O código em C/C++ que irá utilizar a árvore abstrata e a tabela de símbolos para gerar o código objeto em linguagem *assembly*.
- Um arquivo Makefile com comandos para:

- Gerar o código do analisador léxico a partir do arquivo contendo as especificações para o gerador de analisador léxico (g-v2.l).
- Gerar o código do analisador sintático e construtor da árvore abstrata a partir do arquivo contendo as especificações para o gerador de analisador sintático (g-v2.y).
- Compilar os arquivos gerados e os escritos na linguagem de programação adotada no projeto (C/C++).
- Gerar o executável (O professor deve ser capaz de obter o executável do trabalho digitando apenas o comando *make* em uma *shell* do Linux).

A pasta compactada deve ser submetida como uma tarefa a ser criada no site da disciplina. Os códigos gerados devem estar preparados para executarem no sistema operacional Linux (ambiente onde os trabalhos serão avaliados). O compilador C/C++ utilizado pelo professor para compilar os códigos será o gcc/g++ na versão 14. O Flex utilizado será o de **versão 2.6.4**. O Bison utilizado será o de **versão 3.8.2**.

Importante:

Cópias idênticas ou modificadas dos códigos ou de partes dos códigos são consideradas plágios. **Plágio é crime.** O aluno que cometer plágio em seu trabalho receberá nota zero no mesmo. **SE O COMPILADOR GERADO ESTIVER CORRETO, MAS O ALUNO NÃO O APRESENTAR OU NÃO SOUBER EXPLICAR O TRABALHO, RECEBERÁ NOTA ZERO NO MESMO.**